DESENVOLVIMENTO DE UM MÓDULO DE NAVEGAÇÃO PARA ROBÔS NA CATEGORIA ROBOCUP@HOME

Vinícius Fiorio Casarin (Fundação Hermínio Ometto) vinicius.casarin01@alunos.fho.edu.br
Renato Luciano Cagnin (Fundação Hermínio Ometto) renato_cagnin@fho.edu.br

Resumo

Este trabalho contempla o estudo na área de Inteligência Artificial e robótica, com objetivo de desenvolver um módulo de navegação para um robô autônomo e simulado que cumpra com os requisitos da competição RoboCup@Home. O método proposto realiza a simulação pelo ambiente Gazebo, permitindo testar o comportamento de um robô em diferentes desafios e cenários. Para o mapeamento e percepção, foi utilizado o software RViz2, onde foi empregado um sensor LIDAR e técnicas de SLAM. A junção e coordenação de todos os elementos integrados foram arquitetadas por meio do middleware do ROS 2. A navegação é realizada pelo *Navigation Stack 2*, que utiliza o algoritmo de Dijkstra para o planejamento de rotas, enquanto o movimento físico do robô é executado pelo controlador diferencial (*diff drive*) do Gazebo. A associação de lugares e objetos é feita por pontos objetivos rotulados determinados. Os testes realizados mostram a capacidade do robô de evitar obstáculos e chegar a pontos objetivos conhecidos, enquanto mapeia ambientes desconhecidos. Assim, é possível concluir que, embora ainda não possua interpretação semântica, o projeto contribui para a criação de um robô autônomo e representa uma iniciativa na competição.

Palavras-Chaves: Robótica; Navegação; Inteligência Artificial.

1. Introdução

A Inteligência Artificial (IA) é um campo em rápida expansão, com aplicações em diversas áreas. Sua atuação não se limita apenas na automatização de processos, mas também na capacidade de replicar comportamentos e inteligência humana (WOOLDRIDGE; JENNINGS, 1995). Nesse contexto, inserem-se os agentes, capazes de compreender intenções humanas e agir de forma autônoma para atingir objetivos. Podemos descrever um agente racional como uma entidade que age, percebe o ambiente ao seu redor e consegue tomar decisões coerentes em relação a isso, buscando o melhor resultado esperado e lidando com incertezas (RUSSEL; NORVIG, 2022).

Junto do avanço da IA, temos a robótica e a eletrônica, que procuram aplicar os algoritmos ao mundo físico. Para se locomover, um robô necessita mapear o ambiente por meio de sensores,

ou instrumentos de medição, que funcionam como seus sentidos. Segundo Abu-Mahfouz (2022), medição é o processo de associar um número a uma variável física, ou Quantidade Sob Medição (QUM), como a distância entre o robô e um obstáculo. Com a aquisição de dados do ambiente, o robô calcula respostas coerentes por meio de equações matemáticas. Em robótica, esses cálculos baseiam-se em princípios físicos e vetoriais que descrevem o movimento de corpos e forças atuantes (HALLIDAY *et al.*, 2010).

No meio acadêmico, diversas instituições promovem o avanço da robótica autônoma. Entre elas, destaca-se a competição RoboCup@Home, que impulsiona o desenvolvimento de tecnologias de IA voltadas a ambientes residenciais (HART *et al.*, 2024). Para isso, é comum o uso de simulações em software, especialmente no Gazebo, que permitem testar e validar algoritmos com maior praticidade.

Os robôs desenvolvidos na RoboCup@Home devem interagir com humanos para receber comandos e objetivos. Para isso, usam algoritmos, como os *Large Language Models* (LLMs), capazes de compreender instruções em linguagem natural e adquirir novos conhecimentos. Também são usados algoritmos de Reconhecimento de Imagem que obtêm informações enquanto o robô percorre o ambiente (XI *et al.*, 2023). Após definir seu objetivo, o robô realiza o mapeamento do espaço por meio de sensores, possibilitando ao módulo de navegação traçar rotas seguras. Esses caminhos são uma sequência de pontos no espaço geométrico que um robô seguirá. Já o módulo de controle executa o movimento físico, calculando os efeitos dos torques sobre a configuração do robô por meio de funções matemáticas (RUSSEL; NORVIG, 2022).

Robôs dessa área são empregados em tarefas domiciliares, suporte a idosos e automação de tarefas cotidianas, gerando grande impacto social. Sendo isto impulsionado pelo avanço da IA, estima-se que atinja 4,8 trilhões de dólares até 2033 (UN TRADE & DEVELOPMENT, 2025). Tendo isso em vista, este projeto propõe construir o módulo de navegação com potencial para integrar um sistema completo e futuramente participar da RoboCup@Home, representando a instituição Fundação Hermínio Ometto (FHO). A motivação deste projeto vem da possibilidade de contribuir com uma iniciativa científica em robótica para a instituição FHO e por este ser um campo de muito aprendizado.

O projeto tem como objetivo desenvolver o módulo de navegação para um robô simulado que cumpra os requisitos e tenha capacidade de atuar na categoria RoboCup@Home. O foco principal é desenvolver a funcionalidade de planejamento de trajetória, assim como sua capaci-

dade de locomoção e mapeamento. O módulo deve ser capaz de traçar rotas seguras, evitando colisões com obstáculos, para chegar a pontos objetivos diversos em ambiente simulado.

Para concretização deste objetivo, serão seguidos os seguintes objetivos específicos: (i) construir o corpo do robô com sensores e capacidade de locomoção no simulador Gazebo; (ii) construir e integrar o sistema ROS 2 para a comunicação dos softwares, componentes e algoritmos; (iii) desenvolver o módulo responsável pelos movimentos de locomoção do robô; (iv) realizar mapeamento com SLAM e sensores LIDAR no ambiente simulado RViz2; (v) aplicar algoritmos de navegação e planejamento de trajetória com Nav2; (vi) desenvolver modelo para definir pontos objetivos rotulados em diferentes cenários; (vii) criar cenários para validação do sistema.

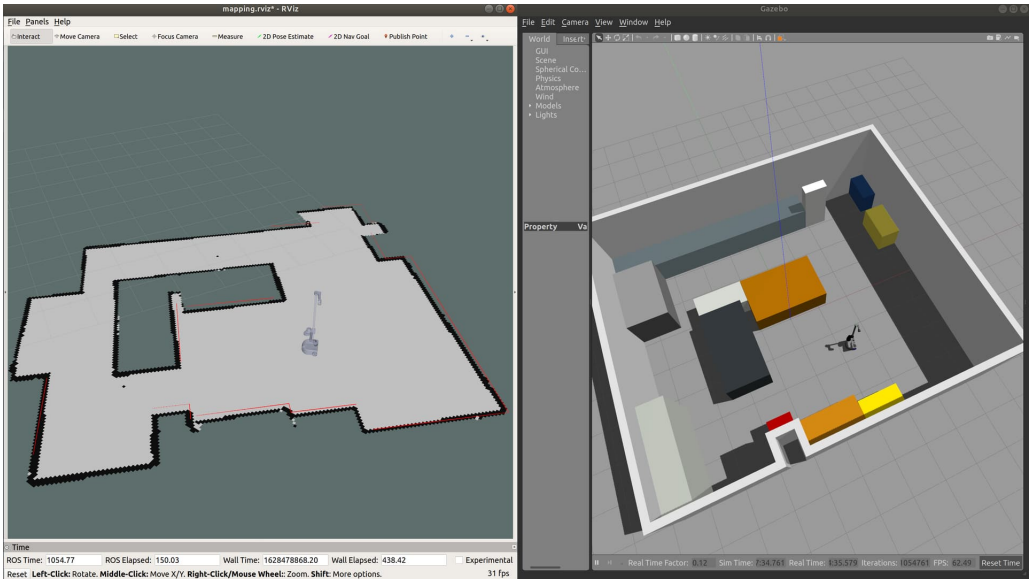
2. Revisão bibliográfica

Neste tópico estão descritos os conceitos e ferramentas usados no desenvolvimento deste trabalho.

2.1. Ferramentas para simulação

O Gazebo é uma ferramenta de simulação 3D para robótica com suporte a física, sensores e atuadores virtuais. Ele permite a construção de mundos virtuais, modelagem de robôs e objetos, além da importação de modelos com sensores, como o LIDAR (GAZEBO, 2024). O RViz (*ROS Visualization*) é uma ferramenta para ROS (*Robot Operating System*) que complementa o Gazebo, sendo utilizada para a interpretação das informações publicadas pelos tópicos do sistema, como leituras de sensores, mapas, trajetórias e frames de referência. Além da visualização de dados de navegação, o RViz permite mapeamento em tempo real, realizado por sensores LIDAR integrados a algoritmos de SLAM, exibindo a construção do ambiente enquanto o robô se desloca na simulação. Essa junção possibilita observar simultaneamente a simulação do comportamento e percepção do robô. A Figura 1 mostra um mesmo cenário exibido pelo RViz (esquerda) e pelo Gazebo (direita).

Figura 1 – Simulação no ambiente RViz e Gazebo



Fonte: https://forum.hello-robot.com/t/is-there-a-way-to-run-funmap-on-gazebo-simulation/288

A integração com o ROS atua como núcleo de processamento e tomada de decisão, permitindo o controle completo do robô, o processamento de dados sensoriais e a execução de algoritmos de navegação, mapeamento e controle, de forma escalável e distribuída. Tudo em um sistema que reúne componentes padronizados e desacoplados (QUIGLEY *et al.*, 2009).

A arquitetura do ROS usa um modelo de sistemas computacionais distribuídos, com uma infraestrutura formada por nós, que são programas com tarefas específicas, responsáveis, por exemplo, pela leitura de sensores e pelo controle de motores. Eles compartilham dados por meio de inscrição em Tópicos usando o sistema *publisher-subscriber* (NEWMAN, 2017).

A comunicação entre os nós é feita através do RMW (ROS Middleware), promovendo coordenação entre componentes. Para isso, o nó se inscreve em Tópicos e define funções de *Callback*, executadas automaticamente quando há novas publicações (NEWMAN, 2017)

2.2. Mapeamento com SLAM

A técnica SLAM (*Simultaneous Localization and Mapping*) é uma abordagem clássica utilizada na robótica, que permite a localização por sensores (como o LIDAR) e mapeamento de forma autônoma da entidade em um ambiente desconhecido. Assim, enquanto percorre seu trajeto, o agente pode construir um mapa do ambiente enquanto simultaneamente determina objetivos, obtém dados computacionais e auxilia na tomada de decisões (ZHANG; BONG; BELTRAME,

2024).

Neste projeto, o SLAM é usado para criar um mapa do ambiente em tempo real, usando a detecção de obstáculos dos sensores LIDAR. Ele também usa informações dos sensores e do ambiente para ajustar a posição estimada do robô, eliminando o desvio progressivo causado pelos erros de movimentação, corrigindo erros de odometria acumulados e melhorando a estimativa de pose (posição e orientação) do robô no mapa. A Figura 2 ilustra um exemplo da posição de um robô em um mapa que foi gerado a partir de sensores e algoritmos de mapeamento, onde em cinza estão os obstáculos e paredes mapeados por ele.

Figura 2 – Mapeamento de ambiente usando sensores



Fonte: RUSSEL; NORVIG, Inteligência Artificial: Uma Abordagem Moderna, 2022

2.2.1 Sensor LIDAR

O sensor LIDAR (*Light Detection and Ranging*) é um dispositivo de alta precisão amplamente utilizado para mapeamento. Ele funciona a partir de um emissor de pulsos de laser e na detecção do retorno de tempo de voo desses pulsos após atingirem objetos no ambiente. A tecnologia mede o intervalo de tempo entre a emissão e o retorno do pulso para calcular a distância até o objeto, proporcionando dados detalhados sobre a profundidade e a forma do ambiente (XI *et al.*, 2023).

Em contexto de robótica, este sensor é usado para medir a distância entre o robô e obstáculos à sua volta, evitando colisões e danos a ele (GAZEBO, 2024). Também, utilizar um

conjunto destes sensores promove o reconhecimento de cenários e objetos, gerando mapas por nuvem de pontos que fornecem coordenadas precisas ao robô, tornando possível tomar decisões de projeção de trajetória (XI *et al.*, 2023).

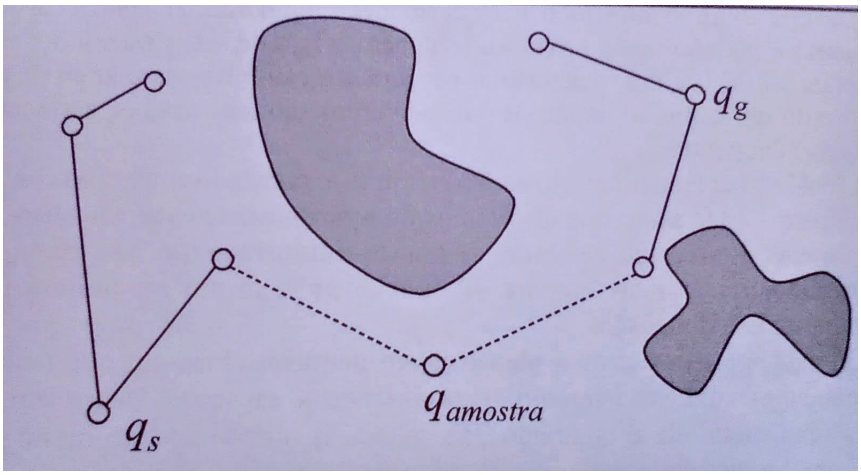
O uso de sensores LIDAR neste projeto permite obter dados de distância entre o robô e os obstáculos ao seu redor, fornecendo informações de percepção espacial e detecção de obstáculos para o SLAM, permitindo o mapeamento do ambiente, e para o planejamento de trajetória, para evitar colisões e traçar rotas.

2.3. Planejamento de trajetória

A Navegação para um robô refere-se ao conjunto de técnicas e processos que permitem ao robô se mover de maneira autônoma em um ambiente para alcançar um objetivo, enquanto evita obstáculos e cumpre tarefas específicas. Na robótica, políticas de movimentação são tratadas como funções de navegação, pois permitem ao robô lidar com erros e imprecisões de movimento de forma mais robusta que um simples planejador determinístico (RUSSEL; NORVIG, 2022).

Os algoritmos de planejamento de trajetória podem ser divididos em duas abordagens: baseada em busca ponto a ponto, onde se verifica a existência de um caminho viável $q_{amostra}$ entre a entidade q_s e seu objetivo q_g, que não possua obstrução, como ilustrado na Figura 3; e baseado em representação global, onde inicialmente não é considerado o objetivo e a posição da entidade, o foco é definir o espaço de todos os movimentos possíveis e mapear regiões navegáveis do ambiente, para então buscar trajetórias possíveis da entidade até seu objetivo.

Figura 3 – Planejamento de trajetória ponto a ponto



Fonte: RUSSEL; NORVIG, Inteligência Artificial: Uma Abordagem Moderna, 2022

2.3.1 Algoritmo de Dijkstra

O algoritmo de Dijkstra resolve problemas de caminhos mínimos a partir de uma única origem e é aplicado em grafos dirigidos ponderados com pesos de arestas não negativos. Pode-se imaginar esse algoritmo como uma generalização da busca em largura, porém Dijkstra considera custos associados às arestas atribuindo uma fila de prioridades mínimas (CORMEN *et al.*, 2024).

Este projeto utiliza *Navigation Stack 2* (Nav2), um sistema de navegação do ROS 2, que usa o algoritmo de Dijkstra, ou seja, uma abordagem em busca ponto a ponto, para que robôs móveis planejem rotas seguras e sigam trajetórias de forma autônoma em ambientes mapeados. Esse sistema permite o robô seguir trajetórias planejadas e alcançar objetivos de posição e orientação.

2.4. Controlador cinemático

O controle cinemático é uma forma de controle que se baseia nas relações geométricas e de velocidade entre as partes do robô e o ambiente, para descrever e comandar o movimento de um robô sem considerar massa, inércia, atrito ou torque. Seu principal objetivo é calcular as velocidades necessárias para que um robô mude sua posição e orientação de um ponto para outro (NEWMAN; 2017).

Para movimentação, o controle do robô foi realizado através do plugin *diff drive* do próprio Gazebo, escolhido por sua compatibilidade direta com Nav2 e por centralizar o controle de locomoção em um único tópico, além de calcular a pose final. Utilizando o modelo cinemático de tração diferencial, este plugin utiliza a Velocidade Linear *V*, em m/s, e a Velocidade Angular ω, em rad/s, desejada para mover o robô para calcular e comandar a velocidade angular da roda esquerda ω_L e direita ω_R (ROS, 2021). No cálculo, Equação 1, também são utilizadas medidas do robô, como raio *r* das duas rodas e distância *b* entre as rodas.

Equação 1 – Cálculo das forças angulares

$$\omega_L = \frac{V - \omega \cdot b/2}{r}$$
$$\omega_R = \frac{V + \omega \cdot b/2}{r}$$

Fonte: Autor, 2025

2.5. Competição RoboCup@Home

Na competição RoboCup@Home, os robôs são avaliados em tarefas que simulam atividades domésticas, como buscar objetos, seguir pessoas e preparar receitas culinárias. Os testes ocorrem em cenários estruturados para representar apartamentos reais, contendo móveis e pessoas como obstáculos. A configuração mínima do ambiente inclui um quarto, uma sala de jantar, uma sala de estar e uma cozinha. A autonomia do robô é valorizada na competição, sendo alcançada por meio de algoritmos e técnicas capazes de lidar com diferentes situações (HART *et al.*, 2024).

Neste projeto, o cenário de testes foi projetado com base na configuração mínima. Móveis foram distribuídos de modo a atuar como obstáculos, permitindo avaliar a capacidade do robô em evitá-los, planejar rotas e navegar até eles de forma autônoma utilizando o Nav2.

2.6. Trabalhos relacionados

Como a RoboCup@Home é uma competição, existem projetos relacionados desenvolvidos por diversas instituições, com diferentes focos e técnicas. O conteúdo, técnicas e práticas realizadas são compartilhadas no acervo da categoria por meio de TDPs e pelo GitHub.

Um exemplo nesta modalidade é a equipe Austin Villa@Home, da Universidade do Texas, Estados Unidos. Em seu projeto de controle e navegação, foram utilizados sensores LIDAR para obtenção de dados ambientais. O sistema de navegação foi construído usando o ROS, integrado à arquitetura LAAIR (*A Layered Architecture for Autonomous Interactive Robots)* para planejamento autônomo e em tempo real de rotas. Para o mapeamento semântico, a equipe empregou o PRISM (*Pose Registration for Integrated Semantic Mapping*), que permite ao robô estimar localizações de cômodos e identificação de objetos (HART *et al*, 2018).

Outro exemplo de projeto é o robô da equipe UChile Peppers, que desenvolveu um sistema baseado em Visual SLAM, substituindo o uso do LIDAR devido à sua baixa resolução e curto alcance. O novo método utiliza câmeras RGB-D integradas ao sistema ORB-SLAM modificado, que combina informações visuais e odométricas para realizar a localização e o mapeamento em tempo real. Para a navegação, o robô emprega o ROS, enquanto a interpretação semântica do ambiente é apoiada por um sistema de memória de longo prazo (chamada *Long-Term Memory*), capaz de associar entidades e eventos a informações contextuais, aprimorando a interação e a compreensão do ambiente. As técnicas e tecnologias destas equipes estão representadas na Tabela 1.

Tabela 1 – Comparação entre equipes da RoboCup@Home

Aspecto	Austin Villa@Home	UChile Peppers
Sensores utilizados	LIDAR e câmeras RGB-D	Câmeras RGB-D e sensores de odometria
Algoritmo de Navegação	ROS integrado à arquitetura LAAIR	ORB-SLAM modificado (Visual SLAM), integrado ao ROS
Abordagem Semântica	Mapas por SLAM com informações semânticas pelo PRISM	Long-Term Memory (LTM) que associa entidades e eventos a informações semânticas
Foco Principal	Integração de navegação, percepção e mapeamento semântico em uma arquitetura unificada	Navegação autônoma baseada em visão e aprimoramento cognitivo com memória semântica

Fonte: Autor, 2025

Em comparação com projetos da Austin Villa@Home e da UChile Peppers, este trabalho adota sensores LIDAR para aquisição de dados ambientais e utiliza o ROS 2 com o módulo Nav2, configurado com o planejador Dijkstra para o controle de navegação. O mapeamento é realizado por meio de um SLAM métrico 2D baseado em grade. Não há entendimento semântico, porém o robô usa um sistema simplificado de waypoints para associar coordenadas a objetos e cômodos.

3. Materiais e métodos

A metodologia segue o modelo em espiral, contemplando as etapas de planejamento, modelagem, implementação, testes e validação. Em cada ciclo, foram definidos os algoritmos e materiais, realizados ajustes, implementadas novas funcionalidades e realizadas verificações e validações (V&V) até os testes finais. Por envolver componentes de hardware (simulado) e software, o projeto também adota a abordagem *meet-in-the-middle*, com desenvolvimento conjunto do robô e dos algoritmos. As decisões de hardware seguem a estratégia *bottom-up*, visando facilitar a modularidade e o acoplamento dos algoritmos, evitando ajustes constantes durante o desenvolvimento.

Os softwares utilizados na construção do robô e do cenário incluem o ambiente virtual Gazebo, versão Fortress 7.9.0, e o RViz2 para visualização do mapeamento. A arquitetura do sistema foi construída com ROS 2, versão Humble. Foi utilizada a linguagem de programação Python. Foi utilizado um notebook com Sistema Operacional Ubuntu 22.04.5 LTS (Jammy Jellyfish), com as seguintes especificações: processador AMD Ryzen 5 5500u com placa de vídeo integrada RENOIR e 8Gb de memória RAM.

3.1. Repositório do projeto

Os arquivos desenvolvidos neste projeto estão disponíveis em um repositório GitHub público do autor: https://github.com/ViniciusFCasarin/Modulo_de_Navegacao.

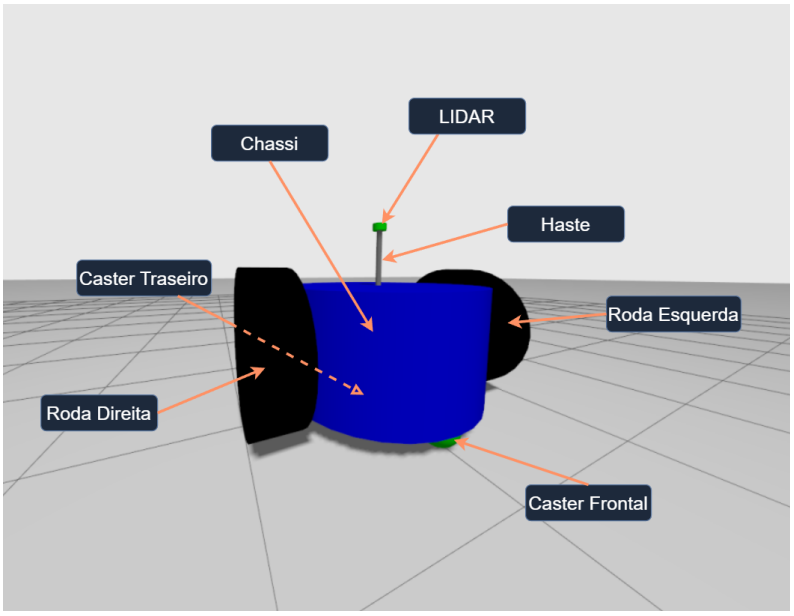
3.2. Etapas de desenvolvimento

Esta seção apresenta de forma detalhada as etapas seguidas durante o desenvolvimento do projeto. As etapas estão organizadas em sequência cronológica das atividades realizadas e compõem os elementos necessários para a navegação do robô.

3.2.1 Construir o robô no ambiente Gazebo

A modelagem do robô, exibido na Figura 4, em ambiente virtual Gazebo foi baseada em robôs existentes na própria competição, porém neste projeto não foram levadas em conta as métricas reais. Portanto, em linguagem SDF, foi construído o Chassi com formato cilíndrico de altura 0,75 metros e raio 0,7 metros. Para locomoção, foram adicionadas 2 rodas laterais idênticas com altura 0,3 metros e raio 0,5 metros. Para ponto de apoio, equilíbrio e estabilidade, foram criadas duas rodas Caster de 0,12 metros, uma frontal, na cor verde, e outra traseira, na cor vermelha. Foi acoplada uma haste de 0,5 metros com sensor cilíndrico de raio 0,05 acima do robô.

Figura 4 – Robô do projeto simulado no software Gazebo



Fonte: Autor, 2025

Os joints (junções) são usados no cálculo da cinemática. Eles conectam o Chassi com as rodas direita e esquerda e com a haste. As rodas Caster também foram conectadas ao Chassi, para isso foi usado um joint especial do tipo "*ball*", permitindo que a roda frontal gire livremente em qualquer direção. Além disso, foi criado um link, chamado base link, para referência principal do robô e outro, chamado footprint, para projeção no solo.

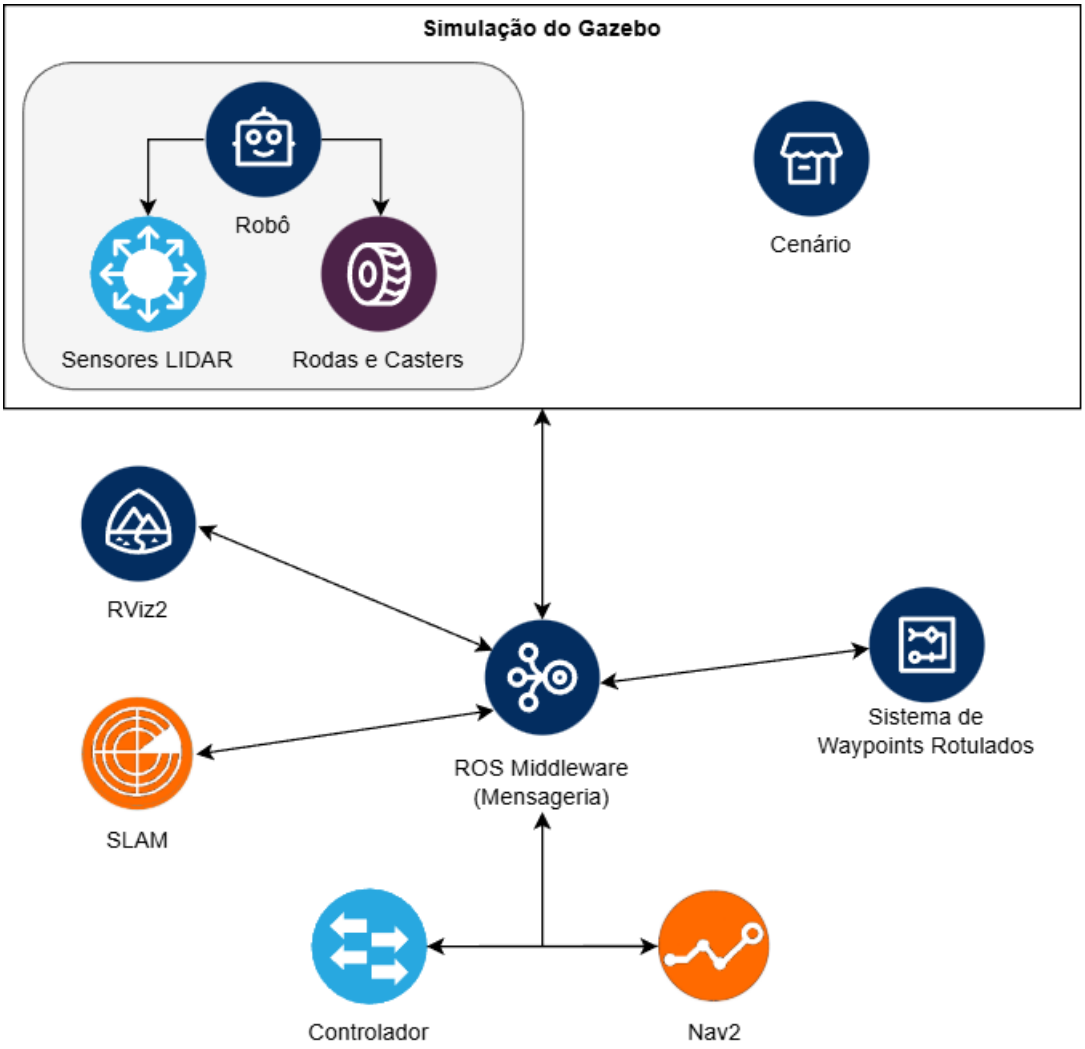
3.2.2 Construir e integrar o sistema ROS 2

Após estabelecida a modelagem do robô, foi construída a arquitetura do ROS 2, que atua como *middleware* e é responsável por interligar e permitir a comunicação entre componentes distribuídos, tanto do robô quanto de softwares. Isso é realizado por meio de um sistema de nós, onde cada um representa um processo independente responsável por tarefas específicas, como traduzir mensagens de um formato para outro, chamado de *bridge* (pontes), e fazer *remapping* (remapeamento) de tópicos para comunicação. Eles se comunicam enviando pacotes de dados por tópicos do sistema de mensageria *publish/subscribe*. A interação entre os componentes é feita de forma assíncrona e desacoplada, por isso os nós utilizam o tempo do Gazebo para manter a sincronização da simulação.

Tendo o ROS 2 como vínculo central na arquitetura do sistema, o RViz2, através do SLAM, consegue projetar onde o robô e o cenário estão, assim como o Nav2 recebe dados

espaciais e de odometria e usa o controlador para dizer ao robô como chegar aos pontos objetivos (Waypoints). A Figura 5 mostra a relação e arquitetura dos componentes do projeto.

Figura 5 – Arquitetura do Sistema



Fonte: Autor, 2025

3.2.3 Desenvolver o módulo de controle

Para locomoção, foi agregado ao robô o *plugin diff drive* do Gazebo, que é responsável por interpretar os comandos de velocidade linear e angular do robô, calcular a rotação de cada roda e publicar sua odometria no tópico odom. Para receber dados, ele é inscrito no tópico cmd_vel, possibilitando que outros componentes controlem o robô.

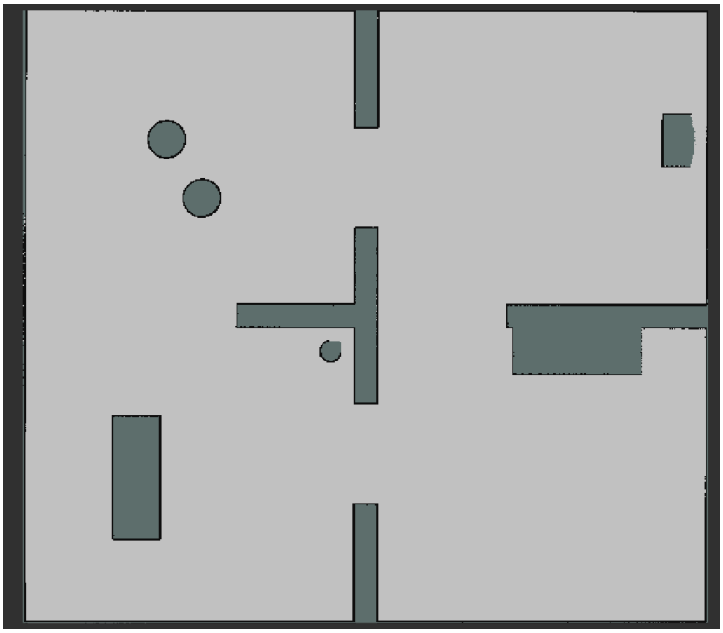
3.2.4 Realizar o mapeamento com SLAM e LIDAR

Foram utilizados sensores LIDAR acoplados ao Chassi para determinar as distâncias do robô aos

objetos à sua volta. Para isso, é utilizado o *plugin sensors*, que cria e publica no tópico nomeado lidar, as distâncias medidas do robô até obstáculos. Como o SLAM e Nav2 esperam receber os dados do sensor em um tópico chamado scan, foi realizado o remapeamento no nó do LIDAR usando uma *bridge* no ROS 2.

O mapeamento desenvolvido segue o método *Grid* usando SLAM *Toolbox*, um pacote de SLAM do ROS 2, onde o mapa é dividido em células, contendo informação se estão ocupadas, representadas pela cor preta, ou vazias, representadas pelas cores azul e branca. O motivo das células vazias serem representadas em duas cores diferentes, é que as mapeadas em azul se referem a uma área livre e explorada, onde o sensor LIDAR não projetou obstáculos, já em branco foi encontrado um obstáculo durante a medição. Para visualização do mapa gerado, informações do sensor, trajetórias traçadas e pose do robô foi utilizado o RViz2, uma ferramenta de visualização 3D do ROS 2 que exibe o robô e o mapeamento do ambiente em tempo real. A Figura 6 mostra um cenário completamente mapeado pelo LIDAR e SLAM.

Figura 6 – Cenário mapeado por SLAM utilizando sensor LIDAR



Fonte: Autor, 2025

Para definir a pose do robô, sensor e obstáculos, são usados frames, ou seja, pontos em um sistema de coordenadas que armazenam a posição e orientação das entidades. O ROS 2 faz uma associação com *Transform Frame* (TF) para rastrear e gerenciar as relações entre os frames para relacionar, por exemplo, a pose do sensor LIDAR em relação ao Chassi do robô.

O mapa gerado pelo SLAM é armazenado pelo tópico *map*, que é usado como referencial

global, mantendo o mapa estável enquanto o robô se desloca. Já para determinar a posição do robô, é usado o tópico odom, que transmite seus dados de odometria, como velocidade linear, angular do robô e posição relativa no espaço. A odometria é associada ao frame *Base Link*, que indica a posição física do robô, neste projeto, foi considerado o Chassi como ponto de referência principal, ou seja, o *Base Link*.

O frame Base Link do robô e o frame de sua odometria, chamada de Odom, começam com a mesma pose, ou seja, os dois estão referenciados no mesmo lugar. A transformação da referência Odom para Base Link é calculada e publicada pelo controlador de acionamento diferencial, diff drive, usando a odometria da roda. Porém, conforme o robô se movimenta no Gazebo, é gerado um erro acumulado, isso ocorre porque a odometria é efetivamente uma medição da velocidade do robô, gerando um acúmulo de medições. Para solucionar isso, foi introduzido um novo frame, chamado de Map, como referência no mundo, enquanto Odom mantém o erro acumulado, o SLAM usa esse erro e calcula a transformação de Map para Odom para corrigir os frames, comparando os registros de scan consecutivos do LIDAR e alinhando os dados para calcular uma pose corrigida. Além disso, ele usa *Loop Closure* para relacionar locais já visitados e redistribuir o erro por toda a trajetória.

3.2.5 Integrar Nav2 ao sistema

A implementação do sistema de navegação Nav2 usou dois componentes principais: o planejamento global e o planejamento local. Eles operam com *costmaps* do ambiente, ou seja, um mapa de custo, gerado pelo pacote Nav2 Costmap 2D, que representa o esforço ou risco associado a navegar por cada célula, diferente do SLAM, onde é gerado um mapa binário. No Nav2, o planejamento global é executado pelo NavFn Planner, que utiliza o algoritmo de Dijkstra para calcular a rota completa entre a posição atual e o objetivo. O algoritmo explora todas as direções, avaliando as células livres até encontrar um caminho de menor custo. Por ser um processo mais pesado, ele é recalculado apenas quando necessário, oferecendo uma visão de longo alcance do ambiente e definindo onde o robô deve ir.

Já o planejamento local é feito pelo DWB Controller, que utiliza o método *Dynamic Window Approach* para controlar o robô. Ele opera sobre o local costmap, com o objetivo de reagir a obstáculos próximos. O Nav2 envia comandos lineares e angulares para o diff drive via tópico cmd_vel.

Em decorrência desse sistema, o robô pode não conseguir alcançar o objetivo desejado. Isso ocorre quando o objetivo está fora dos limites do costmap global, quando está localizado em uma área desconhecida, quando representa uma posição em colisão ou quando simplesmente não existe um caminho viável até ele. Nessas situações, o Nav2 não consegue gerar um plano global válido e interrompe a navegação.

3.2.6 Desenvolver modelo de pontos objetivos rotulados

Foi implementado um sistema simplificado de pontos objetivos nomeados, chamados *waypoints*, que usam um arquivo JSON para associar rótulos a coordenadas de salas e objetos. Além de definir pontos objetivos, este sistema também determina a orientação final do robô. Essa abordagem representa um ponto de partida para a navegação semântica, ao estabelecer uma correspondência entre nomes compreensíveis, como “Kitchen Counter” e “Bedroom Wardrobe”, e posições espaciais específicas, permitindo que o robô se desloque até locais pré-definidos, porém ainda sem capacidade de reconhecimento e entendimento semântico de salas e objetos.

3.2.7 Criar cenários de validação e avaliação

Para visualização, foi definida a rota traçada por uma linha azul e o caminho percorrido por uma linha verde. Foram criados cenários de teste para validar funções do robô e avaliar os seguintes requisitos:

- Estrutura do robô: verificar problemas de locomoção, equilíbrio e erros causados pelo chassi;
- Mapeamento: avaliar se o SLAM mapeou o ambiente corretamente e manteve as referências odométricas;
- Tempo: foi usado um cronômetro para medir o tempo que o robô leva para ir de um ponto inicial a um objetivo;
- Navegação: determinar a capacidade do sistema de traçar rotas de menor caminho do robô ao objetivo, evitar colisões, chegar à posição e orientação objetivo, determinar rotas possíveis e não possíveis, e adaptar-se a ambientes não mapeados.

Para facilitar e possibilitar testes da atuação de sensores e mapeamento, foi implementado

um controle manual de usuário, onde os comandos são publicados via teclado no tópico *keypress*, permitindo operar manualmente a velocidade linear e angular do robô.

4. Resultados

No Gazebo, foram elaborados dois cenários simples e objetivos para validar e avaliar características específicas e desempenho do robô. Também foi criado um cenário aproximado do evento *RoboCup 2024 - @Home League FINALS*, visto na Figura 7, para avaliar o robô em um ambiente de competição.

Figura 7 – Final da RoboCup@Home de 2024

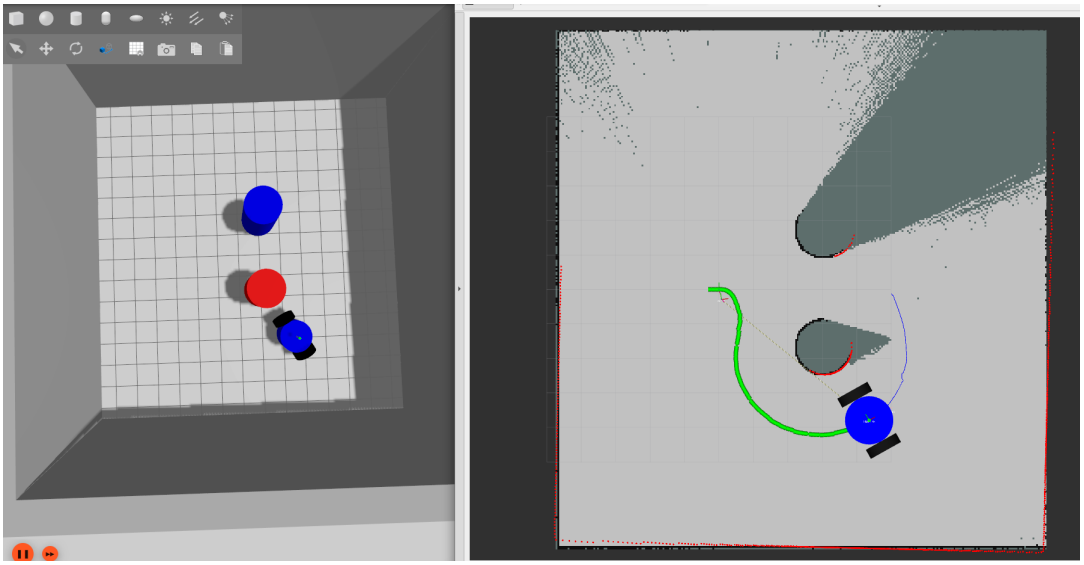


Fonte: https://youtu.be/iuSbg0d9DzQ

4.1. Navegação por obstáculos

No primeiro cenário, Figura 8, o robô deve desviar de dois obstáculos próximos, para avaliar se o robô entende as próprias dimensões e se respeita a margem de segurança para não colidir.

Figura 8 – Teste com obstáculos estáticos

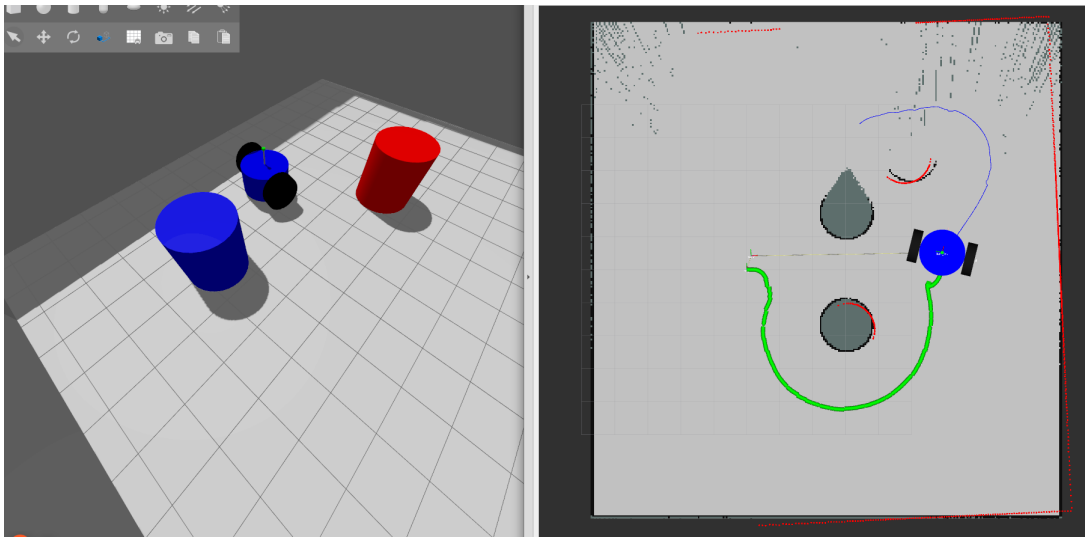


Fonte: Autor, 2025

O robô contornou os 2 obstáculos, sem colidir e por um caminho possível. O tempo decorrido do ponto inicial até o objetivo foi de 26,20 segundos.

Usando este mesmo cenário, também foi testado, na Figura 9, o comportamento do robô com objetos móveis. Para isso, foi alterada a posição do obstáculo azul.

Figura 9 – Teste com obstáculo móvel



Fonte: Autor, 2025

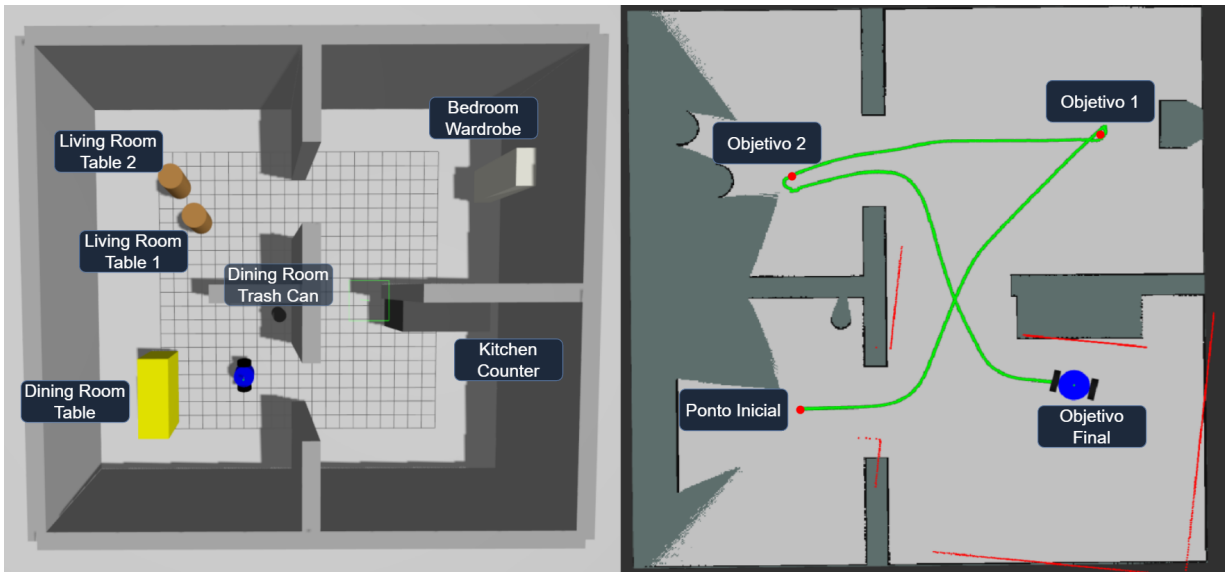
O LIDAR detectou a nova posição do obstáculo, porém o mapeamento manteve o obstáculo na posição passada e na nova.

4.2. Navegação em ambiente doméstico simulado

O terceiro cenário foi dividido em quatro salas e foram definidos os objetivos de destino (x e y) e a orientação (w, em radianos), todos próximos dos móveis. Na Dining Room, foram configurados dois objetivos: Dining Room Table (x = -2,6; y = -1,1; w = -3,1) e Dining Room Trash Can (x = 2,4; y = 1,6; w = 1,57). Na Kitchen, estabeleceu-se o ponto Kitchen Counter (x = 12,9; y = 1,05; w = 1,57). No Bedroom, definiu-se o objetivo Bedroom Wardrobe (x = 14,25; y = 13,3; w = 0,0). Por fim, na Living Room, foram configurados dois destinos: Living Room Table 1 (x = -0,05; y = 10,8; w = -3,14) e Living Room Table 2 (x = -7,5; y = 14,0; w = -0,5).

Aqui é possível avaliar a capacidade de mapeamento com SLAM e LIDAR em um ambiente maior e mais complexo, a velocidade para atingir pontos objetivos rotulados e adaptação a lugares não mapeados e a capacidade do Nav2 de escolher o menor caminho até o objetivo. Na Figura 10 é exibido um teste no cenário no Gazebo (esquerda) e no RViz2 (direita), o primeiro trajeto do robô foi do Ponto Inicial (x = 0,0; y = 0,0; w = 0,0) até o ponto Bedroom Wardrobe, depois para o ponto Living Room Table 1, o qual não estava mapeado e, por fim, foi até o ponto Kitchen Counter.

Figura 10 – Teste de navegação em cenário residencial e simulado



Fonte: Autor, 2025

O SLAM foi capaz de mapear o ambiente e corrigir continuamente os erros de odometria do robô, mantendo sua posição e a dos obstáculos alinhadas no mapa. O LIDAR retornou obstáculos próximos corretamente, porém não atingiu obstáculos distantes, retornando locais vazios e navegáveis. Em ambientes explorados, o Nav2 determinou rotas de menor caminho em

locais visualizados, porém onde não houve varredura do LIDAR, ele determinou que estava fora de alcance. O tempo decorrido do Ponto Inicial até Bedroom Wardrobe foi de 46,78 segundos, deste ponto até Living Room Table 1 foi de 1 minuto e 21,48 segundos e deste ponto até Kitchen Counter foi de 2 minutos e 11,06 segundos.

5. Conclusão

Este trabalho teve como objetivo desenvolver um módulo de navegação para robôs da categoria RoboCup@Home, capaz de planejar trajetórias, mapear ambientes e realizar locomoção autônoma em cenários residenciais simulados. A proposta integrou o sistema ROS 2 com os simuladores Gazebo e RViz2, utilizando o *Navigation Stack 2* (Nav2) para o planejamento de rotas e usando *diff drive* para controle de movimento. O método adotado combinou técnicas de mapeamento com SLAM, detecção de obstáculos por sensores LIDAR e planejamento de trajetória com Nav2, permitindo avaliar o desempenho do robô em cenários da competição.

Com base na seção de resultados, foi possível alcançar o objetivo de construir um sistema coordenado pelo ROS 2 para que um robô simulado consiga: percorrer um ambiente evitando obstáculos usando o controlador diff drive e DWB Controller do Nav2; corrigir erros de odometria acumulados; mapear locais desconhecidos com precisão usando sensor LIDAR e técnica SLAM; traçar rotas seguras e de menor caminho; e alcançar diferentes pontos objetivos. Foi identificado que o sistema não responde adequadamente a obstáculos móveis, pois o mapeamento não reavalia os obstáculos já mapeados. Também, por conta do alcance do sensor LIDAR, o sistema não cria rotas diretas a objetivos muito distantes.

O projeto desenvolvido não possui exploração autônoma nem interpretação semântica do ambiente, limitando-se à navegação em mapas manualmente explorados e a waypoints fixos. A principal dificuldade enfrentada foi a integração das ferramentas utilizadas, por conta da complexidade de comunicação dos componentes.

Contudo, o projeto atual usa somente LIDAR e SLAM para gerar mapas puramente métricos, então eles não carregam significado semântico. Para trabalhos futuros, propõe-se integrar o PRISM, para trazer compreensão semântica do ambiente, e exploração autônoma com Frontier Exploration. Também é proposto integrar outros módulos, como Processamento de Linguagem Natural, permitindo o robô interação natural humano-máquina.

Em síntese, o trabalho apresentou o desenvolvimento de um módulo de navegação ca-

paz de atuar em cenários simulados baseados na RoboCup@Home, mostrando-se viável para aplicações futuras e contribuindo para iniciações científicas nessa área.

REFERÊNCIAS

ABU-MAHFOUZ, Issam. **Instrumentation: Theory and Practice: part 1: principles of measurements**. Pennsylvania State University Harrisburg: The Springer, 2022.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L. *et al.* **Algoritmos: teoria e prática**. 4. ed. Rio de Janeiro: GEN LTC, 2024.

GAZEBO. ***Gazebo Ionic Documentation***. 2024. Página de documentação para o software Gazebo. Disponível em: <https://gazebosim.org/docs/latest/getstarted/>. Acesso em: 9 de jun. 2025.

HALLIDAY, David; RESNICK, Robert; WALKER, Jearl. **Fundamentos de Física**. 9. ed. Volume 1. Rio de Janeiro: LTC, 2010.

HART, Justin W.; MORIARTY, Alexander; PASTERNAK, Katarzyna *et al.* **Robocup@Home 2024: Rule and regulations**. 2024.

HART, Justin W.; SHAH, Rishi; KIRMANI, Sean *et al.* **PRISM: Pose Registration for Integrated Semantic Mapping**. 2018.

NEWMAN, Wyatt S. **A Systematic Approach to Learning Robot Programming with ROS**. 2017.

QUIGLEY, Morgan; GERKEY, Brian; CONLEY, Ken *et al.* **ROS: an open-source Robot Operating System**. 2009.

ROS. **Wiki ROS**, 2021. Página de documentação para o software ROS. Disponível em: <https://wiki.ros.org>. Acesso em: 12 jun. 2025.

RUSSEL, S. J.; NORVIG, P. **Inteligência Artificial: Uma Abordagem Moderna**. 4. ed. São Paulo: Elsevier, 2022.

UN TRADE & DEVELOPMENT. **TradeAI market projected to hit $4.8 trillion by 2033, emerging as dominant frontier technology**. 2025. Disponível em <https://unctad.org/news/ai-market-projected-hit-48-trillion-2033-emerging-dominant-frontier-technology>. Acesso em: 10 abr. 2025.

WOOLDRIDGE, Michael J.; JENNINGS, Nicholas R. **Intelligent agents: theory and practice**. 1995.

XI, Zhiheng; CHEN, Wenxiang; GUO, Xin *et al.* **The Rise and Potential of Large Language Model Based Agents: A Survey**. 2023.

ZHANG, Rongge; BONG, Haechan M., BELTRAME, Giovanni. **Active Semantic Mapping and Pose Graph Spectral Analysis for Robot Exploration**. 2024.